

CSE 4113 – Internet Programming Lab

Project Report

Hopper

Team: Four-Eyed Raven

Submitted By

Team Member	Student ID
TODO: Add member 1 name	TODO: Add student ID
TODO: Add member 2 name	TODO: Add student ID
TODO: Add member 3 name	TODO: Add student ID
TODO: Add member 4 name	TODO: Add student ID

TODO: Add batch information

Department of Computer Science & Engineering

University of Dhaka

Submitted On

TODO: Add submission date

Contents

Project Links	3
1 Introduction	4
1.1 Problem Definition & Context	4
1.2 Target Users & Use Cases	4
1.3 Core Features	4
1.4 Tech Stack Overview	5
2 System Architecture	6
2.1 High-Level Architecture Diagram	6
2.2 Frontend Architecture	8
2.3 Backend Architecture	9
2.4 Database Design	10
3 API Documentation	12
3.1 API Design Overview	12
3.2 Endpoint Reference	13
3.3 Swagger / Postman Collection Link	17
3.4 Sample Request & Response	17
4 Authentication & Security	18
4.1 Authentication Strategy	18
4.2 Authorization & Role Management	18
4.3 Security Measures	19
4.4 Known Vulnerabilities & Mitigations	19
5 Testing & Quality Assurance	19
5.1 Testing Strategy	19
5.2 Test Coverage Summary	19
5.3 Bug Tracking & Resolution Log	19
5.4 Sample Test Cases	19
6 CI/CD & Deployment	19
6.1 Pipeline Overview	19
6.2 Environments	19
6.3 Deployment Steps	19
6.4 Hosting Platform & Live URL	20
6.5 Monitoring & Logging	20
7 Repository & Documentation Quality	20
7.1 Branching Strategy & Commit Conventions	20
7.2 README Completeness Checklist	20

7.3	Code Organization / Folder Structure	20
7.4	Local Setup Instructions	20
8	Evaluation & Reflection	20
8.1	Web Performance & Core Web Vitals	20
8.2	Challenges & Solutions	20
8.3	Limitations & Future Work	20
8.4	Lessons Learned	20
8.5	Individual Responsibility	21
A	Screenshots / UI Walkthrough	21

Project Links

Item	URL
Public Git Repository	https://github.com/CREVIOS/Hopper
Deployed Application URL	https://hopper.farefin.com
API Docs (Swagger/Postman) – public link	https://hopper.farefin.com/api/docs
Demo Video	TODO: Add public demo video URL

1. Introduction

1.1 Problem Definition & Context

Hopper is a self-hosted compute provisioning platform intended for university environments. The repository describes it as a system that slices institution-owned bare-metal resources into isolated runtime environments that can be requested through a web interface, accessed over SSH, and governed through credit-based usage control and single sign-on integration. In the current implementation, the platform exposes these runtimes as Kubernetes-backed pod sessions while preserving the user-facing concept of provisioned virtual machine style workspaces.

The codebase shows that the project addresses several operational problems at once. First, shared academic compute infrastructure requires stronger isolation than ad hoc account sharing or manually provisioned hosts. Second, institutions need a practical way to manage fair usage, including per-session billing and automatic shutdown when credits are exhausted. Third, a university deployment must integrate with centralized identity management rather than maintaining a separate manual authentication system. Hopper therefore combines a browser-based control plane, an API layer, an orchestration service, a relational ledger-backed data model, and Keycloak-based authentication in one deployable platform.

1.2 Target Users & Use Cases

The verified user model in the repository includes at least three application roles: student, professor, and admin. These roles appear in the documented Keycloak realm configuration and are reflected in frontend routes and backend authorization logic. The system therefore serves both end users who consume compute sessions and privileged users who supervise access, credits, and platform status.

Students are the primary runtime users. Confirmed use cases include signing in through the authentication flow, viewing available plans, creating and terminating compute sessions, monitoring CPU and memory usage, opening browser-based development access, using SSH-related settings, and reviewing their credit balance and usage history. Professors and administrators have additional operational and oversight use cases, including allocating credits, inspecting node or platform statistics, reviewing users, and accessing administrative workflows exposed through dedicated backend and frontend routes.

1.3 Core Features

The repository confirms that Hopper provides a multi-service compute management workflow rather than a static informational website. Core platform features include authenticated session management through Keycloak, plan-based runtime creation, automated credit checks before provisioning, orchestration of pod lifecycle events through a Go service, and persistent tracking of sessions and ledger operations in PostgreSQL with

TimescaleDB support. These features are reinforced by the documented API routes, backend services, database migrations, and deployment manifests.

The frontend and backend code also confirm several user-facing capabilities beyond simple provisioning. The platform includes real-time metrics delivery through Server-Sent Events, browser-based terminal and file-management related functionality, administrative dashboards, SSH key management, notifications, and issue-reporting support. The infrastructure and documentation further show support for ingress-based web delivery, TLS termination, Keycloak-backed login and logout, NATS-based event propagation for billing and lifecycle messages, and Kubernetes deployment resources for a full cluster-based installation.

1.4 Tech Stack Overview

The frontend is implemented with SvelteKit 2, Svelte 5, TypeScript, and Vite. This choice is suitable because the project needs route-driven server rendering, interactive dashboards, and a structured component model for authenticated application pages. The dependency set and source tree also show use of lightweight Svelte stores and derived state rather than a heavyweight external state library. In practice, state management is handled through Svelte writable stores for concerns such as authentication, pod state, and notifications, together with route-level server load functions and local derived state in components.

The backend is implemented as a FastAPI application in Python 3.12, backed by Pydantic settings, SQLAlchemy, Alembic migrations, and supporting libraries for SSE, authentication, rate limiting, and asynchronous I/O. This stack is suitable because it supports typed API development, automatic OpenAPI documentation, asynchronous integration with external services, and relatively clear separation of routers, services, models, and middleware. The repository also contains a Go 1.23 orchestration service that uses gRPC and Kubernetes client libraries, which is appropriate for cluster-facing lifecycle management because the native Kubernetes tooling in Go is mature and directly aligned with the platform's orchestration requirements.

The primary database layer is PostgreSQL with TimescaleDB. This combination is suitable because Hopper needs conventional relational storage for users, sessions, roles, and accounting data, while also benefiting from time-series capabilities for metrics aggregation. Messaging and event distribution are handled through NATS JetStream, which fits the platform's need for lightweight asynchronous communication between the orchestrator and the API gateway, especially for billing and metrics events. Authentication is provided by Keycloak using OIDC flows, which is suitable for institutional single sign-on integration and role-based access control.

The deployment and hosting configuration is centered on Kubernetes, Nginx Ingress, and declarative infrastructure artifacts under `k8s/`, `charts/`, and `infrastructure/`. This is suitable because the platform provisions compute sessions as cluster workloads and de-

depends on ingress routing, service discovery, and namespace-level policy enforcement. The repository also confirms supporting infrastructure such as Docker and Docker Compose for local services, cert-manager for TLS certificate automation, and Cilium-enforced network policy definitions for zero-trust traffic control. Optional outbound email support is configured through Brevo or SMTP, but the exact production mail configuration remains environment-dependent.

2. System Architecture

2.1 High-Level Architecture Diagram

The deployed system is organized as a browser-facing frontend, a Python API gateway, a Go orchestration service, an identity provider, a relational database layer, and an event bus. The ingress configuration under `k8s/deploy/03-ingress.yaml` shows that external traffic is split across four path families: `/` for the frontend, `/api` for the gateway, Keycloak realm/resource paths for authentication, and a dedicated code-server proxy path for browser-based development access. The API gateway then coordinates synchronous database work, gRPC calls to the orchestrator, and asynchronous event handling over NATS.

At runtime, the orchestrator is the component that directly manages Kubernetes resources. The FastAPI gateway does not create pods itself; instead it creates or updates the application-facing database state, invokes the orchestrator over gRPC, and consumes follow-up events such as billing, pod lifecycle changes, and metrics updates. PostgreSQL stores the authoritative relational state for users, sessions, the credit ledger, notifications, queue entries, and other application records, while TimescaleDB features are used where available for metrics retention and hypertable support.



The diagram illustrates the system architecture with the following components and interactions:

- Browser UI** (light blue box) sends a **page load / a 202 queue or 201 pod response** to the **SwiftKit routes** component.
- SwiftKit routes** (light blue box) contains **+apiUrl()** and sends **SSE / notifications + /pods/{id}/metrics** to the **Browser UI** and **FastAPI routers**. It also receives **terminal / VS Code WebSocket** from the **Browser UI**.
- FastAPI routers** (light blue box) contains **+Depends()** and receives **202 queue or 201 pod response** from the **SwiftKit routes**. It sends **metrics** to the **SwiftKit routes** and **billing + metrics + pod + namespaces** to the **NATS subjects** component.
- FastAPI routers** sends **CreatePod / terminate pod / restart pod** to the **Go orchestrator** component.
- Go orchestrator** (light blue box) contains **gRPC** and sends **create / watch / terminate** to the **Kubernetes pods** component.
- Go orchestrator** sends **SSH bridge / HTTP proxy** to the **PostgreSQL** component.
- PostgreSQL** (light green box) contains **pod_sessions / ledger** and receives **read / write session state** from the **FastAPI routers**.
- NATS subjects** (light green box) contains **+ metrics metrics.* billing.*** and receives **metrics** from the **FastAPI routers**.
- Kubernetes pods** (light green box) contains **NodePort / port-forward** and receives **create / watch / terminate** from the **Go orchestrator**.

The main operational flow shown in Figure 2 is also verified directly in code. The frontend issues requests through the `/api` path, the gateway checks identity and credits, the

scheduler either reserves capacity or enqueues the request, the orchestrator creates or terminates the Kubernetes workload, and asynchronous follow-up state reaches the UI through SSE streams, notification events, and interactive proxies.

2.2 Frontend Architecture

The frontend is a SvelteKit application organized around file-based routes in `frontend/src/routes`. Route-specific server load functions such as `+layout.server.ts`, `dashboard/+page.server.ts`, `admin/+page.server.ts`, and `Pods/[id]/+page.server.ts` fetch authenticated data on the server side before rendering pages. The helper `frontend/src/lib/api/server.ts` resolves whether those server-side requests should go through an ingress-relative `/api` path or directly to an internal API base URL, depending on environment configuration.

Application-wide authentication state is coordinated in `+layout.server.ts` and `+layout.svelte`. The layout loader attempts `/auth/me` using the `session_token` cookie, falls back to `/auth/refresh` when necessary, and returns the authenticated user and balance to the shell layout. The authenticated shell then renders the sidebar, topbar, notification bell, and balance badge, and it also starts periodic refresh and balance polling timers in the browser. This indicates a mixed SSR-plus-client-refresh strategy rather than a purely client-side single-page architecture.

State management remains lightweight and is implemented with built-in Svelte mechanisms instead of an external library such as Redux or Zustand. The repository defines writable stores in `frontend/src/lib/stores/` for authentication, pod-related state, and notifications. Route components also use local derived state extensively, as seen in multiple `+page.svelte` files. This means the application follows a layered state strategy: server-loaded page data for initial render, writable stores for cross-component reactive state, and local derived values for view-level calculations.

The API layer is split between server-side and browser-side helpers. On the browser side, `frontend/src/lib/api/client.ts` wraps `fetch`, automatically includes credentials, retries once after a silent refresh on non-login 401 responses, and exposes a small SSE helper. On the server side, the route loaders call the backend directly through `apiUrl()`. This division is suitable for a SvelteKit application because it allows SSR data fetching, cookie forwarding, and browser reactivity without duplicating request logic in every route.

The real-time and interactive frontend features are implemented explicitly rather than implicitly. The notification store opens an EventSource connection to `/api/notifications/stream`, the pod detail page opens an EventSource to `/api/pods/{id}/metrics`, and the terminal component opens a WebSocket to `/api/pods/{id}/terminal`. Browser-based editor access is routed through the backend proxy path rather than by direct pod exposure. The repository therefore confirms three distinct frontend interaction modes: SSR page loading, credentialed REST-style API calls, and long-lived SSE or WebSocket channels.

2.3 Backend Architecture

The backend is implemented as a FastAPI-based API gateway in `services/api-gateway/app/main.py`. Its router registration shows a domain-oriented separation by capability: authentication, pods, credits, admin, settings, SSH keys, files, issues, notifications, and usage each live in their own router modules. Shared concerns are applied through dependency injection and middleware, including database sessions from `get_db()`, authentication from `get_current_user()`, audit logging, CORS control, and rate limiting. This structure is closer to a thin-router plus service-module design than to a heavy layered controller-service-repository stack.

The gateway is not a passive CRUD layer. It coordinates synchronous HTTP work with asynchronous background processes and remote orchestration. The `pods` router, for example, validates access, checks user credits, enforces per-user limits, invokes admission logic in `vm_scheduler` and `vm_queue`, persists `PodSession` state, and then calls the Go orchestrator through the async gRPC wrapper in `orchestrator_client.py`. If synchronous admission is not possible, the same router returns a queued response instead of failing immediately. This demonstrates an application-service style workflow centered in router handlers and helper service modules.

Several backend services are event-driven. During application startup, the gateway connects to NATS, starts the billing consumer, and starts the metrics consumer. The billing consumer processes `billing.deducted` and `billing.exhausted` events, applies idempotent credit deductions, manages grace-period behavior, and triggers user notifications. The metrics consumer stores periodic metrics snapshots in the database. The notification service persists rolling user notifications and also subscribes to pod lifecycle events so that `pod.started`, `pod.stopped`, and `pod.failed` can be reflected in the user interface. The result is a hybrid backend architecture combining request-response APIs, background event processing, and stateful session updates.

The admission queue is an especially important architectural feature. The service `vm_scheduler.py` elects a single leader through the `scheduler_leader` table and runs a first-come, first-served admission loop that considers cluster capacity computed from orchestrator node data plus current `PodSession` reservations. The queue service persists queued requests in `vm_queue_entries`. This means queueing is implemented inside the application database rather than delegated to an external job queue. The code explicitly documents that the loop is head-of-line and does not backfill smaller requests behind a blocked older request.

The Go orchestration service is separated into focused internal packages under `services/orchestrator/internal/`. The inspected file tree confirms packages for billing, gRPC endpoints, event publication, leader election, Kubernetes access, watchers, and pod management. The main process connects to NATS, creates Kubernetes clients, starts the gRPC server, and then runs singleton work that subscribes to events, reconciles watcher state, and publishes metrics. In other words, the orchestrator is the cluster-facing runtime

component, whereas the FastAPI application remains the externally facing control plane.

The current backend does not implement a dedicated repository abstraction layer. Database queries are performed directly with SQLAlchemy `AsyncSession` from routers and service modules. This is an important verified characteristic of the codebase and should not be misrepresented as a repository-pattern implementation. If a stricter persistence abstraction is desired later, that would be future work rather than a description of the current system.

2.4 Database Design

The database design combines normalized relational entities for identity, sessions, credits, and operational metadata with a time-series metrics table that can be promoted to a TimescaleDB hypertable when the extension is available. The foundational migration creates separate tables for users, accounts, transfers, ledger entries, pod sessions, and metrics. Later migrations adapt the original GPU-oriented session model into plan-based VM sessions, add notification and queueing support, and introduce API key and network-group support.

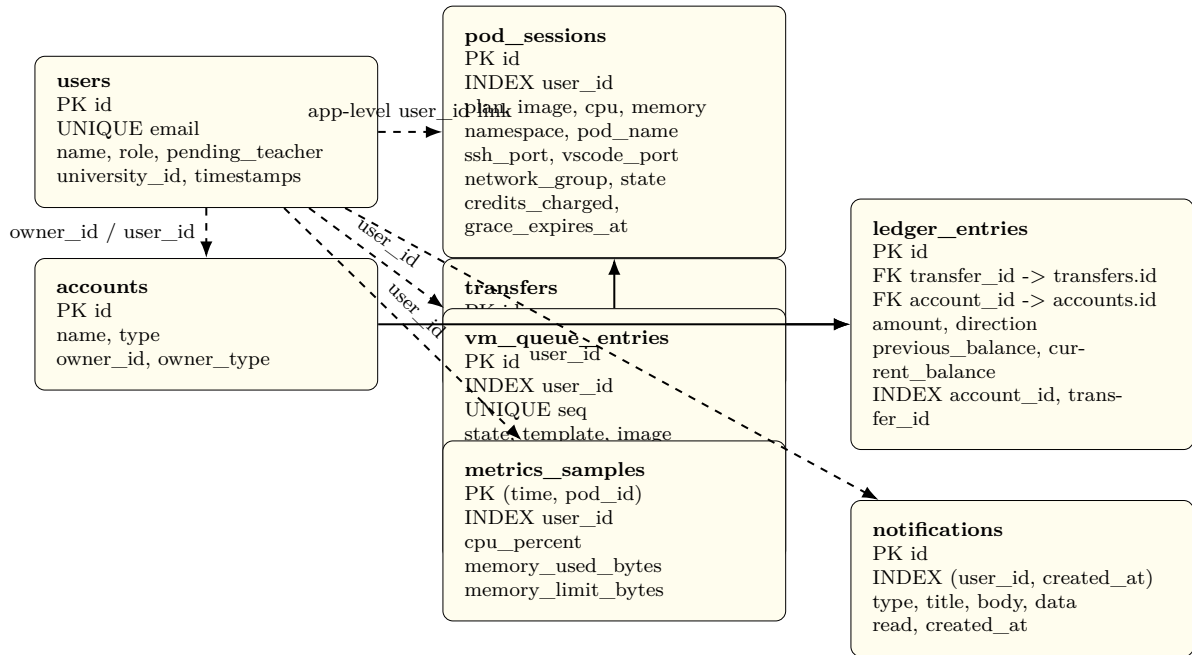


Figure 3: Core entity relationships derived from SQLAlchemy models and Alembic migrations. Solid arrows indicate database-enforced foreign keys; dashed arrows indicate application-level links stored by identifier only.

Figure 3 highlights an important characteristic of the current schema: not every logical relationship is enforced with a foreign key. The strongest database-enforced relationships are in the credit ledger, where `ledger_entries.transfer_id` references `transfers.id`, `ledger_entries.account_id` references `accounts.id`, and `vm_queue_entries.pod_session_id` references `pod_sessions.id`. By contrast, many user-owned records such as `pod_sessions`,

`notifications`, and `api_keys` store a `user_id` without a declared SQL foreign key. Ownership is therefore enforced primarily through application logic and authenticated route checks rather than through a dense relational constraint graph.

The credit subsystem is intentionally normalized into three tables: `accounts`, `transfers`, and `ledger_entries`. This supports double-entry accounting and avoids storing mutable balances as the only source of truth. The initial migration also creates indexes on `ledger_entries.account_id` and `ledger_entries.transfer_id` and uses PostgreSQL rules to prevent updates or deletes on ledger rows and transfers. That design decision is consistent with the service code, which treats the ledger as append-only and uses transaction-level locking to serialize deductions.

The VM lifecycle subsystem centers on `pod_sessions`. Each row stores the user-facing VM identity, selected plan, image, CPU and memory limits, namespace and pod name, exposed ports, current state, accumulated credits, and optional network-group data. Queueing is separated into `vm_queue_entries`, which freezes the requested plan and resolved image at enqueue time and adds a monotonically increasing `seq` field. The migration creating this table also defines a partial index on queued rows by sequence, which supports efficient first-come, first-served head-of-line admission scans.

The notification and metrics subsystems are also specialized rather than overloaded into a generic event table. Notifications form a rolling per-user window backed by a composite index on (`user_id`, `created_at`) because the main queries are recent notifications and unread counts. Metrics are stored in `metrics_samples` with a composite primary key (`time`, `pod_id`) and a separate index on `user_id`. The migration attempts to create a Timescale hypertable and retention policy when the database supports those functions, but it explicitly degrades gracefully to a regular PostgreSQL table when TimescaleDB is unavailable.

Several auxiliary tables exist outside the core ERD and contribute to operational completeness: `audit_logs`, `ssh_keys`, `issue_reports`, `email_codes`, `scheduler_leader`, and `api_keys`. These support auditing, SSH management, issue reporting, email verification and reset flows, leader election for the scheduler loop, and scoped programmatic access. Because Section 2 focuses on the main architectural data paths, they are described here textually rather than all being included in one overloaded ERD figure.

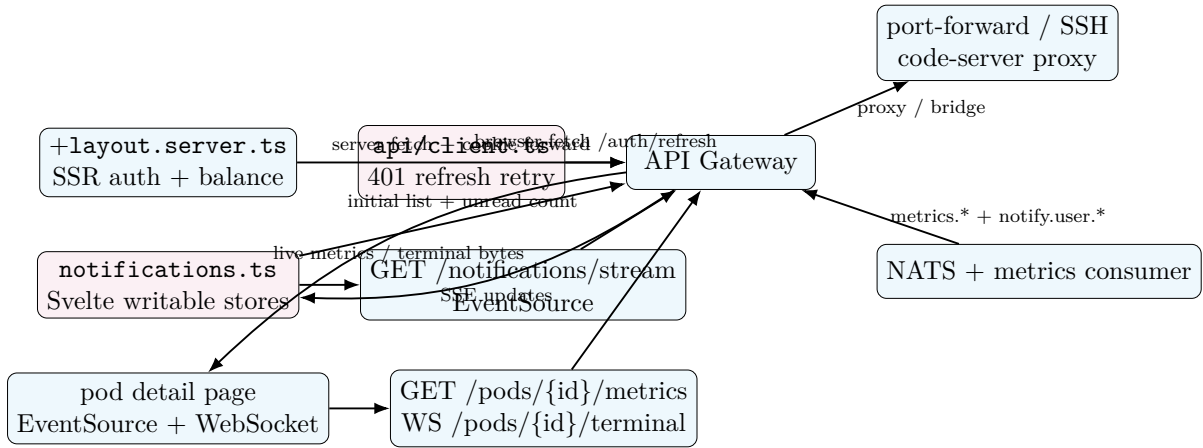


Figure 4: Verified automatic UI data-sync mechanisms: SSR refresh, SSE streams, and WebSocket proxying.

The repository confirms that automatic UI synchronization is implemented in multiple ways rather than through one generic push channel. As shown in Figure 4, the layout loader refreshes authentication state and forwards cookies server-side, the browser-side API client retries after silent refresh on eligible 401 responses, notifications are pushed over an `EventSource` stream, pod metrics are streamed through a separate `EventSource`, and terminal or browser-based editor access is proxied through WebSocket or HTTP upgrade paths in the gateway. This is a concrete, code-backed real-time strategy. It should therefore be documented as a combination of SSR data refresh, SSE fan-out, and backend-mediated interactive proxies rather than as a generic polling-only or WebSocket-only system.

3. API Documentation

3.1 API Design Overview

Hopper exposes a primarily REST-style HTTP API through the FastAPI gateway in `services/api-gateway/app/main.py`. The frontend API client hard-codes `/api` as its public base path, and the Kubernetes ingress rewrites that path before forwarding requests to the gateway service. Internally, the application routers are mounted directly at prefixes such as `/auth`, `/pods`, `/credits`, and `/admin`; externally, users reach them as `/api/auth`, `/api/pods`, `/api/credits`, and so on. The application metadata declares API version 0.1.0, but the repository does not implement URI-based versioning such as `/v1`. The current versioning approach is therefore documentation-level rather than path-level.

The dominant request and response format is JSON over HTTPS. FastAPI and Pydantic models are used for request validation and typed response shaping, for example `LoginRequest`, `SignupRequest`, `CreatePodRequest`, `PodResponse`, `CreditBalanceResponse`, and `UserResponse`. The frontend client sets `Content-Type: application/json`

automatically for non-form submissions and includes browser credentials on every request. A smaller set of endpoints uses alternative transports where the feature requires them: file upload uses `multipart/form-data`, file download returns `application/octet-stream`, notification and metrics feeds use Server-Sent Events (SSE), and terminal or browser-editor sessions use proxied WebSocket or HTTP upgrade paths.

Authentication is enforced through the shared dependency `get_current_user()`. The main browser flow uses secure `HttpOnly` cookies, especially `session_token` and `refresh_token`, which are issued after successful Keycloak-backed sign-in. Programmatic access is also supported through Hopper API keys supplied either in `X-API-Key` or as `Authorization: Bearer hop_....`. The code explicitly restricts `read_only` API keys to `GET`, `HEAD`, and `OPTIONS` requests, while API-key creation and revocation require a real cookie-backed session. Authorization is role-aware rather than endpoint-global: the repository verifies at least three roles, namely `student`, `professor`, and `admin`.

Validation and error handling are a combination of schema validation and route-level checks. Pydantic enforces field structure, lengths, and constrained values, while the routers enforce application rules such as credit sufficiency, ownership, allowed domains, password policy, and role restrictions. The common error format is FastAPI's JSON `{"detail": "..."} payload`, although validation failures inherited from framework-level schema errors may return the standard structured 422 detail list. The codebase also shows consistent use of meaningful HTTP status codes such as 201 Created, 202 Accepted, 204 No Content, 401 Unauthorized, 402 Payment Required, 403 Forbidden, 404 Not Found, 409 Conflict, 422 Unprocessable Entity, and 429 Too Many Requests.

The API does not follow one universal pagination contract, but several repeatable conventions are implemented. Credit history and audit-log style listings use `limit` and `offset` query parameters. Notification and issue listing endpoints impose bounded maximums, and issue administration accepts a simple `status` filter. Usage endpoints accept a `range` parameter with confirmed values `1h`, `6h`, `24h`, and `7d` to control aggregation windows. These conventions are pragmatic and code-backed, but they are not wrapped in a separate global pagination abstraction at the current stage of the project.

3.2 Endpoint Reference

The report focuses on the important public application endpoints that are intended for user-facing or evaluator-facing interaction. Internal readiness and liveness handlers such as `/healthz` and `/readyz` are deliberately excluded from the main table because they are deployment-facing operational probes rather than end-user APIs.

Authentication and identity endpoints

Method	Route	Purpose	Auth	Authorized Role
GET	/api/auth/login	Starts the OIDC login flow with PKCE, state, and nonce, then redirects the browser to Keycloak.	No	Public
POST	/api/auth/signup	Creates a local-account sign-up request with role selection and password-policy checks.	No	Public
POST	/api/auth/verify-email	Confirms an emailed verification code.	No	Public
POST	/api/auth/resend-code	Re-sends a verification or password-reset code.	No	Public
POST	/api/auth/forgot-password	Starts the password-reset flow for an existing account email.	No	Public
POST	/api/auth/reset-password	Resets a password after code validation and, where applicable, marks the email verified.	No	Public
POST	/api/auth/login	Performs direct email-password login and issues session cookies on success.	No	Public
GET	/api/auth/callback	Completes the OIDC authorization-code exchange after Keycloak redirects back.	No	Public callback endpoint
POST	/api/auth/refresh	Uses the refresh-token cookie to mint a new access cookie.	Refresh cookie	Session holder
GET	/api/auth/me	Returns the authenticated user's current profile and pending-teacher status.	Yes	Any authenticated user
POST	/api/auth/logout	Revokes the refresh token where possible, clears cookies, and redirects to Keycloak logout.	Yes	Any authenticated user
POST	/api/auth/api-keys	Creates an API key and returns the plaintext key exactly once.	Yes	Any authenticated user with session cookie
GET	/api/auth/api-keys	Lists the caller's API keys without revealing stored secrets.	Yes	Any authenticated user
DELETE	/api/auth/api-keys/{key_id}	Revokes one of the caller's API keys.	Yes	Any authenticated user with session cookie

Pod lifecycle and interactive session endpoints

Method	Route	Purpose	Auth	Authorized Role
GET	/api/pods/plans	Returns the plan catalogue exposed to the provisioning UI.	No	Public
GET	/api/pods/	Lists the current user's pod sessions.	Yes	Any authenticated user
POST	/api/pods/	Creates a pod synchronously when capacity is available, or returns 202 Accepted with queue information when the request is enqueued.	Yes	Any authenticated user; network_group limited to professor/admin
GET	/api/pods/availability	Reports whether synchronous capacity is currently available together with queue length.	Yes	Any authenticated user
GET	/api/pods/queue	Lists the caller's active queue entries and queue positions.	Yes	Any authenticated user

Method	Route	Purpose	Auth	Authorized Role
DELETE	/api/pods/queue/{entry_id}	Cancels one of the caller's still-queued VM requests.	Yes	Queue-entry owner
GET	/api/pods/{pod_id}	Returns details of a specific pod session.	Yes	Pod owner
DELETE	/api/pods/{pod_id}	Terminates a pod session and releases its reserved resources.	Yes	Pod owner
GET	/api/pods/{pod_id}/metrics	Streams live pod metrics to the UI through Server-Sent Events.	Yes	Pod owner
HTTP proxy	/api/pods/{pod_id}/vscode/{path}	Proxies browser-based code-server traffic for an owned pod across GET, POST, PUT, PATCH, DELETE, and OPTIONS.	Yes	Pod owner
WS	/api/pods/{pod_id}/vscode/{path}	WebSocket upgrade path used by the proxied browser editor.	Yes	Pod owner
WS	/api/pods/{pod_id}/terminal	Provides interactive terminal access to a running pod.	Yes	Pod owner

Credits and usage endpoints

Method	Route	Purpose	Auth	Authorized Role
GET	/api/credits/balance	Returns the caller's current credit balance and account identifier.	Yes	Any authenticated user
GET	/api/credits/history?limit={dots}&offset={dots}	Returns the caller's credit ledger history using limit-offset pagination.	Yes	Any authenticated user
GET	/api/credits/teachers	Lists professor accounts with balances for the admin grant workflow.	Yes	Admin only
GET	/api/credits/students	Lists student accounts with balances for allocation workflows.	Yes	Admin or professor
POST	/api/credits/allocate	Grants or reallocates credits, with semantics that differ for admin and professor roles.	Yes	Admin or professor
GET	/api/usage/{pod_id}?range=1h{textbar6h}{textbar24h}{textbar7d}	Returns historical usage series for a specific pod.	Yes	Any authenticated user
GET	/api/usage/summary/me/series?range=1h{textbar6h}{textbar24h}{textbar7d}	Returns aggregated historical usage across all of the caller's pods.	Yes	Any authenticated user
GET	/api/usage/summary/me	Returns a recent usage summary for the caller.	Yes	Any authenticated user

Administration endpoints

Method	Route	Purpose	Auth	Authorized Role
GET	/api/admin/users	Lists all users with role and teacher-approval metadata.	Yes	Admin only
GET	/api/admin/teacher-requests	Lists pending teacher sign-up requests.	Yes	Admin only
POST	/api/admin/teacher-requests/{user_id}/approve	Promotes a pending teacher request to the professor role.	Yes	Admin only
POST	/api/admin/teacher-requests/{user_id}/reject	Rejects a pending teacher request while keeping the user as student.	Yes	Admin only

Method	Route	Purpose	Auth	Authorized Role
GET	/api/admin/courses	Returns the current course list. The implementation is presently an empty proof-of-concept response.	Yes	Admin only
GET	/api/admin/nodes	Returns cluster node capacity and readiness data via the orchestrator.	Yes	Admin only
GET	/api/admin/stats	Returns aggregate dashboard statistics such as total users and active VMs.	Yes	Admin only
GET	/api/admin/active-vm	Lists all currently active VM sessions together with owner details.	Yes	Admin only
PATCH	/api/admin/users/{user_id}/role	Changes a user's role with safeguards against invalid or dangerous demotions.	Yes	Admin only
GET	/api/admin/audit-logs?limit=\dots&offset=\dots	Returns recent audit-log entries using limit-offset pagination.	Yes	Admin only

Files, SSH keys, settings, notifications, and issue tracking

Method	Route	Purpose	Auth	Authorized Role
PUT	/api/settings/vscode	Accepts a VS Code settings blob. The current implementation acknowledges the request but still documents future persistence work in comments.	Yes	Any authenticated user
GET	/api/ssh-keys/	Lists the caller's SSH public keys.	Yes	Any authenticated user
POST	/api/ssh-keys/	Adds an SSH public key after format, duplicate, and per-user quota checks.	Yes	Any authenticated user
DELETE	/api/ssh-keys/{key_id}	Deletes one of the caller's SSH keys.	Yes	Key owner
GET	/api/files/{pod_id}/list	Lists files inside a running pod directory.	Yes	Pod owner
POST	/api/files/{pod_id}/upload	Uploads a file to a running pod using multipart form submission and SCP behind the gateway.	Yes	Pod owner
GET	/api/files/{pod_id}/download	Downloads a file from a running pod as an octet-stream attachment.	Yes	Pod owner
POST	/api/issues/	Creates a user issue report, optionally linked to a pod.	Yes	Any authenticated user
GET	/api/issues/me	Lists issue reports submitted by the current user.	Yes	Any authenticated user
GET	/api/issues/admin?status=\dots	Lists issue reports for administrative triage, optionally filtered by status.	Yes	Admin only
POST	/api/issues/admin/{issue_id}/resolve	Marks an issue report as resolved.	Yes	Admin only
GET	/api/notifications/	Returns recent notifications together with an unread count.	Yes	Any authenticated user
POST	/api/notifications/read-all	Marks all of the caller's notifications as read.	Yes	Any authenticated user

Method	Route	Purpose	Auth	Authorized Role
POST	/api/notifications/{notification_id}/read	Marks one notification as read.	Yes	Notification owner
GET	/api/notifications/stream	Streams live notifications to the browser using Server-Sent Events.	Yes	Any authenticated user

3.3 Swagger / Postman Collection Link

The FastAPI gateway exposes the default Swagger UI, and the repository's ingress configuration makes it reachable publicly at <https://hopper.farefin.com/api/docs>. This URL is verified from the combination of the gateway's default FastAPI documentation settings and the ingress rule that forwards the external `/api` prefix to the backend service.

No Postman collection URL was found in the inspected repository. The report therefore retains that gap explicitly: **TODO: Add a public Postman collection URL if the team publishes one later.**

3.4 Sample Request & Response

Example 1: direct login request

The following example reflects the request body accepted by `LoginRequest` in `services/api-gateway/app/schemas/user.py`. On success, the handler also sets secure `HttpOnly` session cookies; those headers are omitted here for brevity and because tokens must not be reproduced in a report.

Listing 1: Representative login request to the Hopper API

```
curl -X POST "https://hopper.farefin.com/api/auth/login" \
  -H "Content-Type: application/json" \
  -d '{
    "email": "student@example.edu",
    "password": "[REDACTED]"
  }'
```

Listing 2: Representative login response body

```
{
  "id": "2f3ac4d8-7d3d-4f7b-a2df-0e4f07a8d421",
  "email": "student@example.edu",
  "name": "Example Student",
  "role": "student"
}
```

Example 2: pod creation request

This second example shows a provision request based on the verified `CreatePodRequest` and `PodResponse` schemas. The route may return 201 `Created` for immediate admission or 202 `Accepted` when the request is queued; the JSON below illustrates the immediate-creation path.

Listing 3: Representative pod creation request

```
curl -X POST "https://hopper.farefin.com/api/pods/" \
  -H "Content-Type: application/json" \
  -H "Cookie: session_token=[REDACTED]" \
  -d '{
    "plan": "small",
    "template": "ubuntu"
  }'
```

Listing 4: Representative pod creation response body

```
{
  "id": "9bc3d6ee-4b7a-4482-a9de-4948fdde2ce1",
  "user_id": "2f3ac4d8-7d3d-4f7b-a2df-0e4f07a8d421",
  "state": "running",
  "plan": "small",
  "image": "hopper/vm-ubuntu:22.04",
  "cpu": "1",
  "memory": "2Gi",
  "node_name": "worker-1",
  "namespace": "hopper-user-2f3ac4d8",
  "ssh_port": 32022,
  "vscode_port": 32080,
  "ssh_password": "[REDACTED]",
  "network_group": null,
  "created_at": "2026-07-19T12:00:00Z",
  "updated_at": "2026-07-19T12:00:30Z"
}
```

4. Authentication & Security

4.1 Authentication Strategy

TODO: Describe the verified authentication flow and why it was chosen.

4.2 Authorization & Role Management

TODO: Document the confirmed application roles and access-control model.

4.3 Security Measures

TODO: Summarize verified validation, secrets handling, rate limiting, network, and transport protections.

4.4 Known Vulnerabilities & Mitigations

TODO: Add verified known risks, mitigations, and remaining gaps from the repository or test docs.

5. Testing & Quality Assurance

5.1 Testing Strategy

TODO: Summarize the verified unit, integration, end-to-end, and other automated test layers.

5.2 Test Coverage Summary

TODO: Add measured coverage or coverage-report references after verification.

5.3 Bug Tracking & Resolution Log

TODO: Summarize verified bug-tracking artifacts such as BUG_TRACKING_LOG.md.

5.4 Sample Test Cases

TODO: Add representative verified test cases from the current suites.

6. CI/CD & Deployment

6.1 Pipeline Overview

TODO: Describe the verified CI and delivery workflow from repository automation and scripts.

6.2 Environments

TODO: Document the confirmed development, test, staging, and production environments if they exist.

6.3 Deployment Steps

TODO: Summarize the verified deployment procedure from current infrastructure files.

6.4 Hosting Platform & Live URL

TODO: Add the actual hosting platform and public live URL after verification.

6.5 Monitoring & Logging

TODO: Describe the verified monitoring, logging, and observability components.

7. Repository & Documentation Quality

7.1 Branching Strategy & Commit Conventions

TODO: Add the actual branching and commit practices only after verifying repository history or team policy.

7.2 README Completeness Checklist

TODO: Audit the README against setup, environment, run/test, and usage expectations.

7.3 Code Organization / Folder Structure

TODO: Summarize the verified repository structure and service boundaries.

7.4 Local Setup Instructions

TODO: Condense the verified local setup steps for reproducibility.

8. Evaluation & Reflection

8.1 Web Performance & Core Web Vitals

TODO: Add measured Lighthouse or browser-tool performance metrics later.

8.2 Challenges & Solutions

TODO: Add verified implementation or deployment challenges and how they were addressed.

8.3 Limitations & Future Work

TODO: Add realistic limitations and future improvements based on verified project scope.

8.4 Lessons Learned

TODO: Summarize team-level lessons after the full report content is drafted.

8.5 Individual Responsibility

TODO: Replace the placeholder rows with actual team-member responsibilities and contribution evidence.

Member Name	Core Responsibilities & Feature Ownership	Key PRs / Commit Contributions	Share (%)
TODO: Add name	TODO: Add responsibilities	TODO: Add PRs or commits	TODO: Add %
TODO: Add name	TODO: Add responsibilities	TODO: Add PRs or commits	TODO: Add %
TODO: Add name	TODO: Add responsibilities	TODO: Add PRs or commits	TODO: Add %
TODO: Add name	TODO: Add responsibilities	TODO: Add PRs or commits	TODO: Add %
TODO: Add name	TODO: Add responsibilities	TODO: Add PRs or commits	TODO: Add %
TODO: Add name	TODO: Add responsibilities	TODO: Add PRs or commits	TODO: Add %
TODO: Add name	TODO: Add responsibilities	TODO: Add PRs or commits	TODO: Add %
TODO: Add name	TODO: Add responsibilities	TODO: Add PRs or commits	TODO: Add %

A. Screenshots / UI Walkthrough

TODO: Add annotated screenshots, UI flows, and captions using image files placed under `report/figures/`.

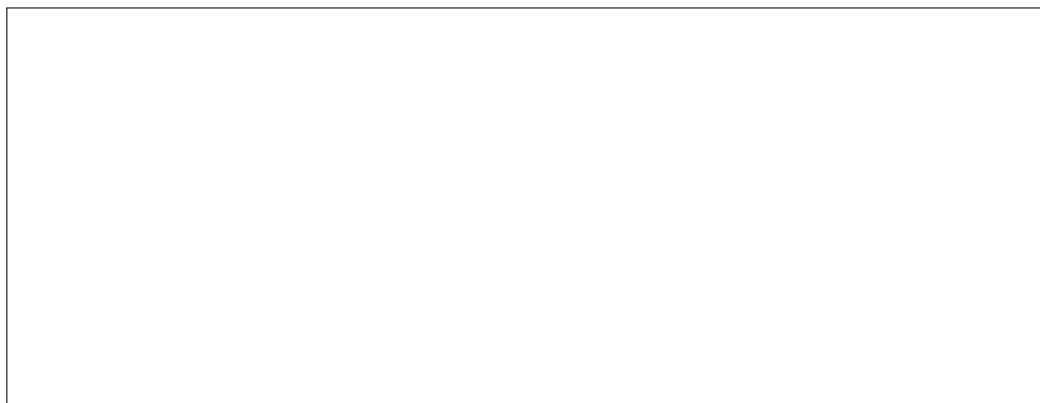


Figure 5: Placeholder for a UI walkthrough screenshot loaded from `report/figures/`.